Changing K2cloud config.ini Settings

Technical Note

Date: 30 January 2025

Version: 1.0

Table of Contents

1. Config.ini Overrides in Fabric 8.1 at K2cloud Space Creation Time 3

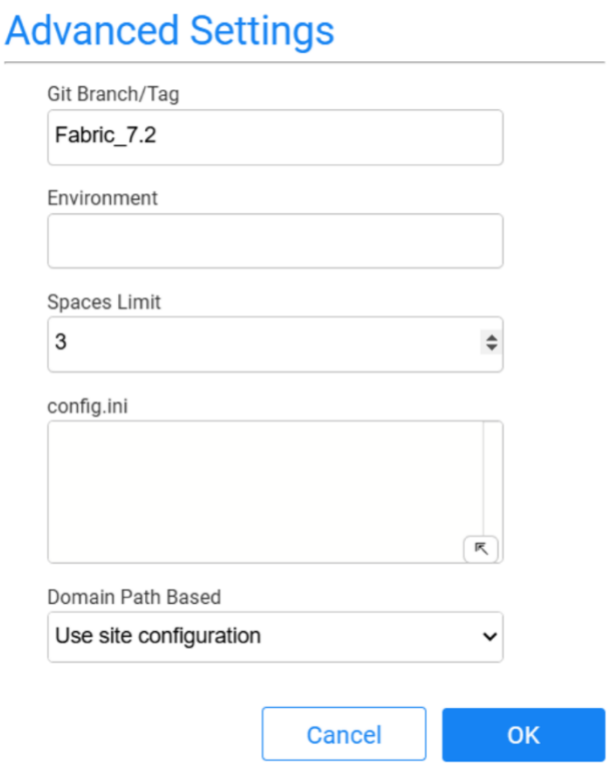
2. Config.ini Updates on a Created Space 5

2.1. Using the config_update.sh Script to Update config.ini 5

2.2. Using Kubernetes Secrets to Update Configuration 8

1. Config.ini Overrides in Fabric 8.1 at K2cloud Space Creation Time

Starting with **Fabric 8.1**, users can override config.ini settings when creating **a space**. This capability is available in the **Project Advanced Settings** section, where you can specify custom config.ini values to replace parameters from the default config.ini provided in the Fabric Docker image.



Key Points:

1. **Creation-Time Only:**
- Config overrides can only be applied **during space creation**.
- After a space is created, the config.ini settings cannot be modified through the K2cloud Orchestrator interface. The space profile and its configuration remain fixed.

The following section describes how you can achieve this using one of two methods.

2. **Purpose of Overrides:**
- The overrides allow you to customize certain config.ini parameters to your project's specific needs. These adjustments are useful for tailoring settings such as API schema, ports, keyspace creation, and more.

3. **Limitations:**
- Not all config.ini values can be overridden via the Project Advanced Settings.
- Parameters that are restricted from being overridden are pre-defined in the space profile. These restrictions ensure system integrity and consistency across spaces.

4. **Example of non-overridable Parameters:**

Below are parameters specified in the space profile that **cannot** be overridden by the Advanced Setting.

```
fabric|OVERRIDE_API_SCHEMA|HTTPS|ADD
fabric|OVERRIDE_API_PORT|443|ADD
fabric|CREATE_KEYSPACE_FOR_LU|true|ADD
fabricdb|MDB_DEFAULT_CACHE_PATH|/opt/apps/fabric/pod_tmp/fdb_cache
default_pubsub|TYPE|MEMORY
common_area_pubsub|TYPE|MEMORY
fabric|WEBSERVER_FILTERS|[{\"class\":\"com.k2view.cdbms.ws.ProxyForward\",\"patterns\":[\"/
studio/*\"],\"params\":{\"target\":\"http://localhost:3000\",\"isStaticTarget\":true}},{\"c
lass\":\"com.k2view.cdbms.ws.ProxyForward\",\"patterns\":[\"/socket.io/*\"],\"params\":{\"t
arget\":\"http://localhost:3000\",\"isStaticTarget\":false}}];
fabric|ENABLE_DB_INTERFACE_PROXY|true
default_session|RECONNECT_MAX_DELAY_MS|1000
fabric|ENABLE_BROADWAY_DEBUG_SERVLET|true
fabric|WEB_SESSION_EXPIRATION_TIME_OUT|540
fabric|SERVER_AUTHENTICATOR|block_all
```

5. **Caveats:**

Restarting Fabric services using the K2cloud Orchestrator's Pause | Resume is required for specific settings to take effect.

2. Config.ini Updates on a K2cloud Created Space

There are two methods for doing so

- Updating config.ini on every node
- Performing updates using the Kubernetes secret.

2.1. Using the config_update.sh Script to Update config.ini

The config_update.sh script, located at /opt/apps/fabric/script in the Fabric image, allows you to modify specific sections and keys in the config.ini file. This script supports a variety of operations, such as adding, updating, or removing key-value pairs.

Note: This procedure must be performed on **every node** in the cluster, as changes are not automatically propagated.

Pre-Requisites

To run the script:

1. Connect to the Fabric container:

   docker exec -it [spacename]-fabric bash

2. Navigate to the script directory:
   cd /opt/apps/fabric/script

3. Use the config_update.sh script

```
#=============================================================================================================
========
USAGE="
Usage: $(basename "$0") [option] \n
Update key value in specific section of the config file\n
Options: \n
    -h | --help                         Shows the usage \n
    -s | --section  [Section_name]      Section name to update in the config File Section name [section] \n
    -k | --key      [Key_name]          Key in config to Update \n
    -v | --value    [Key_value]         Value to set (as single word with no space)\n
    -a | --add                          Will add a new key to section (Will not protect from duplicates). \n
    -c | --comment                      Will comment out a specified key \n
    -r | --remove                       Will remove a specified key from the file \n
    -l | --list     <update list>       File with list of configs to update, each line in file format [section]|[key]
|[value]|<{ADD|}>. \n
    -f | --file     [config_file_path]  Config file path to update \n
Example: \n
./$(basename "$0") -s [section] -k [key] -v [value_with_no_space] -f [config file]
"
#=============================================================================================================
=========
```

Options

- **-h | --help**: Displays the usage guide.
- **-s | --section [section_name]**: Specifies the section in config.ini to modify.
- **-k | --key [key_name]**: Identifies the key within the section to modify.
- **-v | --value [key_value]**: Sets the value for the specified key (single word, no spaces).
- **-a | --add**: Adds a new key-value pair to the specified section. (Duplicate entries are not prevented.)
- **-c | --comment**: Comments out the specified key in the configuration file.
- **-r | --remove**: Removes the specified key from the file.
- **-l | --list [file_path]**: Updates multiple configurations from a file. Each line should be formatted as [section][key][value].
- **-f | --file [config_file_path]**: Specifies the path to the config.ini file.

Key Features

1. **Section-Specific Modifications**: Target specific sections in config.ini for precise updates.
2. **Flexible Operations**: Options to add, update, remove, or comment out keys.
3. **Batch Updates**: Use a list file for bulk modifications across multiple keys and sections.

Important Notes

- This is a **manual procedure** and does not automatically synchronize changes across nodes.
- After making changes, ensure proper restart of the Fabric environment (e.g., via **K2cloud Orchestrator** or Kubernetes scaling).
- Changes in the config.ini file may require a pod restart to take effect.

Consideration: Configuration Drift

While the ability to manually update configurations on individual nodes using the config_update.sh script provides flexibility, it introduces several challenges related to **configuration drift**:

The Dual Nature of Local Config Changes

- **Solution:** Local modifications allow teams to adapt configurations for specific use cases or testing needs without affecting the rest of the cluster.
- **Problem:** These local changes may create a disconnect between:
  1. The configuration maintained centrally (e.g., in **K2cloud Orchestrator** or Kubernetes Secrets).
  2. The local configurations on individual Fabric nodes.

This mismatch can lead to inconsistencies and unexpected behaviors across the cluster.

Challenges of Config Drift

1. **Mismatch Between Secrets and Nodes**:
   - When scaling up, new pods or nodes will use the configuration defined in the **Secret** rather than the locally modified configuration.
   - This discrepancy results in nodes running different versions of the configuration, potentially breaking cluster operations or introducing hard-to-troubleshoot issues.

2. **Lack of Synchronization Across Nodes**:
   - Changes made on one node do not propagate to other nodes in the cluster.
   - Teams must manually replicate changes across all nodes, increasing the likelihood of human error and further exacerbating config drift.

3. **Manual Effort for Each Node**:
   - Every manual change requires logging into each Fabric node and applying updates individually. This process is time-consuming and error-prone, especially in environments with many nodes.

Recommendations to Mitigate Config Drift

1. **Centralize Configurations**:
   - Use **K2cloud Orchestrator** or **Kubernetes Secrets** to centralize and manage configurations.
   - Avoid making local manual changes unless necessary.

2. **Automate Propagation**:
   - Automate configuration changes across all nodes in the cluster using tools like kubectl or a cluster-wide configuration management process.
   - For example, modify the Kubernetes Secret and use a **scale-down/scale-up** approach to ensure consistency.

3. **Regular Reconciliation**:
   - Periodically verify and reconcile node-level configurations with the central configuration source.
   - Use tools like kubectl diff or scripting solutions to compare and identify drifts.

4. **Document Local Changes**:
   - Keep a record of any manual changes made to individual nodes for reference during troubleshooting or when scaling the cluster.

5. **Rolling Restarts**:
   - If a change must be applied to all nodes, consider a **rolling restart** to ensure nodes restart with the updated configuration from the central source.

2.2. Using Kubernetes Secrets to Update Configuration

Kubernetes provides a secure and flexible mechanism for managing configuration and sensitive data through **Secrets**. Below is a detailed explanation of how to use Kubernetes Secrets to update the configuration for Fabric.

Overview of Kubernetes Secrets

- A **Secret** is a Kubernetes object that securely stores sensitive information, such as configuration values, certificates, or credentials.
- Secrets are mounted as files or environment variables into the pods and are kept encrypted in the cluster.

When using a Kubernetes Secret to update the configuration, the process involves:

1. **Fetching the existing Secret.**
2. **Decoding and modifying its content.**
3. **Re-encoding and applying the updated Secret.**
4. **Scaling down and scaling up the environment to apply changes.**

Steps to Update Configuration Using Kubernetes Secrets

1. Fetch the Secret

To fetch the Secret named config-secrets in YAML format:

kubectl get secrets config-secrets -n [namespace] -o yaml

- kubectl: The Kubernetes command-line tool.
- get secrets: Command to retrieve Secret objects.
- config-secrets: Name of the Secret object.
- -n [namespace]: Specifies the namespace containing the Secret.
- -o yaml: Outputs the Secret in YAML format.

The output includes a data section where sensitive values are Base64-encoded.

2. Decode the Secret

Decode the Base64-encoded content to view or edit it:

echo "<BASE64_ENCODED_STRING>" | base64 -d

This reveals the configuration values in a key-value format. For example:
fabric:MDB_DEFAULT_SCHEMA_CACHE_STORAGE_TYPE=FILESYSTEM
fabric:MDB_DEFAULT_CACHE_PATH=/opt/apps/fabric/pod_tmp/mdb_cache
default_pubsub|TYPE=MEMORY

3. Modify the Configuration

Update the necessary key-value pairs. Examples of modifiable configurations:

Web Session Timeout:
fabric:WEB_SESSION_EXPIRATION_TIME_OUT|540

4. Base64 Encode

After editing, re-encode the updated content:

```
echo "<UPDATED_CONFIG>" | base64
```

5. Apply the Updated Secret

Edit and apply the changes to the Secret:

```
kubectl edit secrets config-secrets -n [namespace]
```

- This opens the Secret in a text editor.
- Paste the updated Base64-encoded content into the data section.
- Save and exit the editor. Changes are automatically applied.

6. Restart the Environment

For the new configuration to take effect, perform a Kubernetes **scale down** and **scale up**:

```
kubectl scale statefulset <statefulset-name> --replicas=0 -n [namespace]
kubectl scale statefulset <statefulset-name> --replicas=1 -n [namespace]
```

- Scaling down terminates all pods associated with the stateful set.
- Scaling up recreates the pods, applying the updated Secret during startup.

A rolling restart (described below) will take time. During this period, inconsistencies due to configuration changes may surface issues, so you should avoid performing this change. That said, a rolling restart to apply a minor version should be considered for high availability.

Additional Notes

- **Automated Process:** Secrets can be automated during Fabric's initial startup process. Before creating the stateful set, the Secret should be configured and linked to ensure the pods use it throughout their lifecycle.

Fabric vs. Studio Configurations:

- In **Fabric**, configuration changes are applied every time the service starts.
- In **Studio**, configurations are applied during container creation and do not change dynamically.

Following these steps, you can securely manage and update configurations using Kubernetes Secrets, ensuring consistency and security across your cluster.